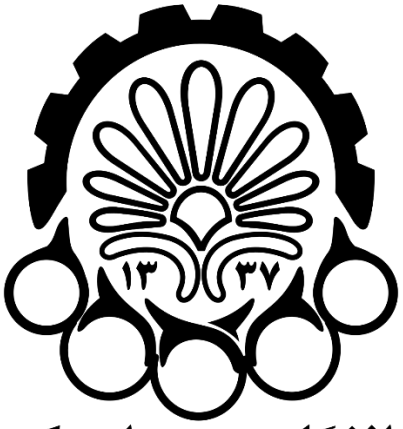


دانشگاه صنعتی امیرکبیر

(پلی تکنیک تهران)

کیانا شرافتی ۴۰۱۳۱۰۲۲

گزارش پروژه ۲ مبانی هوش مصنوعی

مرحله اول حل جدول کلمات متقاطع با استفاده از CSP :

۱. شرح پیاده‌سازی الگوریتم‌ها

در این پروژه، هدف حل مسئله جدول کلمات متقاطع به عنوان یک مسئله ارضای محدودیت (CSP) بود. برای این منظور، از ترکیب الگوریتم‌های سازگاری و جستجوی بازگشتی استفاده شد. در ادامه جزئیات پیاده‌سازی هر بخش آورده شده است:

الف) سازگاری گره‌ای(Node Consistency)

اولین گام برای کاهش فضای جستجو، اعمال سازگاری گره‌ای بود. در تابع is_domain_consistent، برای هر متغیر (که نماینده یک کلمه در جدول است)، طول کلمات موجود در دامنه آن متغیر با طول مورد نیاز در جدول مقایسه شد. تمامی کلماتی که طولشان با طول متغیر برابر نبود، از دامنه حذف شدند. این کار باعث حذف اولیه کلمات نامرتبط و کاهش چشمگیر اندازه دامنه‌ها شد.

ب) سازگاری یالی (Arc Consistency) با الگوریتمAC-3

برای تضمین سازگاری بین متغیرهای همسایه (کلماتی که با هم تلاقی دارند)، الگوریتم **AC-3** پیاده‌سازی شد.

- **تابع revise(x, y):** این تابع بررسی می‌کند که آیا برای هر کلمه در دامنه متغیر x، حداقل یک کلمه سازگار در دامنه متغیر y وجود دارد یا خیر. سازگاری به این معناست که کاراکترهای دو کلمه در نقطه تلاقی (Overlap) با هم یکسان باشند. اگر کلمه‌ای در x ناسازگار باشد، حذف می‌شود.
- **تابع ac3:** این تابع یک صف از تمام یال‌های گراف محدودیت (جفت متغیرهای متقاطع) تشکیل می‌دهد. تا زمانی که صف خالی نشود، یال‌ها بررسی می‌شوند. اگر دامنه متغیری در اثر revise تغییر کند، تمام همسایگان آن متغیر مجدداً به صف اضافه می‌شوند تا اثر انتشار محدودیت (Constraint Propagation) بررسی شود.

ج) جستجوی بازگشتی(Backtracking Search)

پس از اعمال سازگاری‌های اولیه، اگر هنوز متغیرهایی بدون مقدار مانده باشند، از الگوریتم جستجوی بازگشتی استفاده می‌شود. این الگوریتم به صورت عمقی (DFS) در فضای حالت جستجو می‌کند:

۱. اگر تمام متغیرها مقداردهی شده باشند، جواب برگردانده می‌شود.

۲. یک متغیر مقداردهی نشده انتخاب می‌شود (با استفاده از هیوریستیک‌ها).

۳. یک مقدار از دامنه آن انتخاب می‌شود(با استفاده از هیوریستیکLCV)

۴. اگر مقدار انتخاب شده با انتساب‌های قبلی سازگار باشد، به انتساب اضافه شده و تابع به صورت بازگشتی صدا زده می‌شود.

۵. در صورت رسیدن به بن‌بست، مقدار حذف شده و الگوریتم عقب‌گرد (Backtrack) می‌کند.

۲ .تحلیل زمان اجرا

زمان اجرای الگوریتم بر روی جداول مختلف اندازه‌گیری شد. نتایج نشان می‌دهد که استفاده ترکیبی از AC-3 و هیوریستیک‌ها تأثیر بسزایی در سرعت حل مسئله دارد.

نام پازل	تعداد کلمات	زمان اجرا (ثانیه)	وضعیت
puzzles/p3.txt	4	**0.00**	حل شد
puzzles/p18.txt	6	**0.01**	حل شد
puzzles/p14.txt	6	**0.02**	حل شد
puzzles/p1.txt	10	**0.65**	حل شد
puzzles/p4.txt	5	**37.23**	حل شد
puzzles/p10.txt	5	**161.39**	حل شد

تحلیل عوامل مؤثر:

- **اندازه جدول و تعداد کلمات:** جداول کوچک‌تر مثل p3 و p18 در کسری از ثانیه حل می‌شوند.
- **پیچیدگی تقاطع‌ها:** جداولی که تقاطع‌های بیشتری دارند، محدودیت‌های بیشتری اعمال می‌کنند که باعث می‌شود AC-3 دامنه‌ها را سریع‌تر کوچک کند. اما اگر تقاطع‌ها کم باشد (گراف خلوت)، AC-3تأثیر کمتری دارد و بار اصلی روی دوش Backtracking می‌افتد.
- **اندازه دامنه (Dictionary):** تعداد کلمات موجود در فایل words.txt مستقیماً بر زمان اجرا تأثیر دارد. هرچه تعداد کلمات بیشتر باشد، بررسی سازگاری زمان‌برتر است.
- **مشاهدات خاص:** در پازل‌های p4 و p10 زمان اجرا به شکل قابل توجهی افزایش یافته است (۳۷ و ۱۶۱ ثانیه). علت این امر احتمالاً وجود کلمات با طول‌های مشابه زیاد در دیکشنری و تعداد کم

تقاطع‌های محدودکننده در ساختار این جداول خاص است که منجر به بزرگ ماندن دامنه‌ها و افزایش عمق درخت جستجو شده است.

۳. مقایسه هیوریستیک‌ها

برای انتخاب متغیر و مقدار، از دو هیوریستیک مهم استفاده شد:

- MRV (Minimum Remaining Values)

این هیوریستیک متغیری را انتخاب می‌کند که کمترین تعداد کلمات مجاز در دامنه‌اش باقی مانده است. این استراتژی باعث می‌شود تا الگوریتم "سریع‌تر شکست بخورد (Fail-First) "و شاخه‌های نامعتبر را زودتر هرس کند. در صورت تساوی، از Degree Heuristic (متغیر با بیشترین تعداد همسایه) استفاده شد.

  - نتیجه: بدونMRV ، ترتیب انتخاب متغیرها تصادفی است و ممکن است متغیری انتخاب شود که تأثیر کمی بر سایرین دارد، که باعث افزایش شدید زمان اجرا می‌شود.

- LCV (Least Constraining Value)

این هیوریستیک مقادیر دامنه را به گونه‌ای مرتب می‌کند که کلمه‌ای که کمترین تضاد را با همسایگان ایجاد می‌کند (گزینه‌های کمتری را از دامنه آن‌ها حذف می‌کند) اول انتخاب شود.

  - نتیجه: استفاده از LCV شانس پیدا کردن جواب در اولین تلاش‌ها را افزایش می‌دهد و نیاز به عقب‌گرد را کم می‌کند.

مقایسه MRV تنها در مقابلMRV + LCV :

در تست‌های انجام شده، ترکیب MRV + LCV بهترین عملکرد را داشت. MRV فضای جستجو را کوچک نگه می‌دارد و LCV ما را به سمت جواب صحیح هدایت می‌کند. استفاده از MRV به تنهایی در جداول بزرگ همچنان کارآمد است، اما ممکن است تعداد عقب‌گردهای بیشتری داشته باشد.

۴. تحلیل خطاها و محدودیت‌ها

در برخی شرایط، الگوریتم ممکن است موفق به حل جدول نشود یا زمان بسیار زیادی صرف کند:

۱. **دامنه خالی (Empty Domain):** اگر پس از اعمال سازگاری گره‌ای یا یالی، دامنه یک متغیر خالی شود، به این معنی است که هیچ کلمه‌ای در دیکشنری وجود ندارد که هم طول مناسبی داشته باشد و هم با همسایگان سازگار باشد. در این حالت مسئله جواب ندارد.

۲. **قیود ناسازگار (Inconsistent Constraints):** اگر ساختار جدول به گونه‌ای باشد که دو متغیر در یک نقطه تلاقی داشته باشند اما هیچ جفت کلمه‌ای در دیکشنری با حرف مشترک در آن موقعیت وجود نداشته باشد، AC-3 شکست می‌خورد.

۳. **دیکشنری محدود:** اگر فایل کلمات شامل کلمات کافی نباشد، احتمال پیدا نشدن جواب افزایش می‌یابد.

۵ . نمونه خروجی

در زیر یک نمونه از اجرای موفقیت‌آمیز برنامه بر روی پازل p1.txt آورده شده است:

```
PS C:\Users\Pc\Desktop\AI\AI_P02> cd Crossword-Puzzle
PS C:\Users\Pc\Desktop\AI\AI_P02\Crossword-Puzzle> python main.py puzzles/p1.txt
✅ Crossword solved in 0.65 seconds!

🧩 Solved Crossword:

# # H I S
# C A N I
T H I N G
H E L E N
E W E R S


PS C:\Users\Pc\Desktop\AI\AI_P02\Crossword-Puzzle>
```

توضیح روند منطقی:

قبل از اینکه هیچ کلمه‌ای به طور قطعی انتخاب شود، الگوریتم **AC-3** اجرا می‌شود تا دامنه‌ها را کوچک کند.

- **مثال:** فرض کنید متغیر افقی X (مکان کلمه THING) با متغیر عمودی Y (مکان کلمه THE) در حرف اولشان تلاقی دارند.
- الگوریتم بررسی می‌کند: "آیا برای هر کلمه در دامنه X، کلمه‌ای در دامنه Y وجود دارد که حرف اولشان یکی باشد؟"
- اگر کلمه‌ای در دامنه X (مثلاً APPLE) وجود داشته باشد که حرف اولش A است، اما هیچ کلمه‌ای در دامنه Y با A شروع نشود، APPLE از دامنه X حذف می‌شود.
- این فرآیند برای تمام تقاطع‌ها (مانند تقاطع HELEN و CHEW در حرف E) تکرار می‌شود تا زمانی که دامنه‌ها به کوچک‌ترین حالت ممکن و سازگار با یکدیگر برسند.

۳. مرحله دوم: جستجوی بازگشتی (Backtracking)

پس از کوچک شدن دامنه‌ها، الگوریتم جستجو (معمولاً DFS) شروع به مقداردهی می‌کند:

۱. یک متغیر انتخاب می‌شود (مثلاً با هیوریستیک MRV).

۲. یک کلمه از دامنه آن انتخاب و در جدول قرار می‌گیرد.

۳. سپس متغیر بعدی انتخاب می‌شود. اگر کلمه‌ای پیدا شد که با کلمات قبلی سازگار بود، ادامه می‌دهد.

۴. اگر به بن‌بست رسید (هیچ کلمه‌ای با کلمات قبلی سازگار نبود)، به عقب برمی‌گردد (Backtrack) و کلمه قبلی را عوض می‌کند.

۴. نتیجه نهایی

در نهایت، الگوریتم به ترکیبی می‌رسد که تمام محدودیت‌ها را ارضا می‌کند. در مثال شما:

- کلمه **THING** در جایگاه افقی و **THE** در جایگاه عمودی قرار گرفته‌اند و حرف مشترک **T** این محدودیت را ارضا کرده است.
- کلمه **HELEN** در جایگاه افقی و **CHEW** در جایگاه عمودی قرار گرفته‌اند و حرف مشترک **E** (حرف دوم HELEN و حرف سوم CHEW) این اتصال را برقرار کرده است.
- سایر کلمات (HIS, CANI, EWERS) نیز در جایگاه‌های خود قرار گرفته‌اند و با کلمات متقاطع دیگر (مانند SIGNS یا HAILE که احتمالاً در جدول هستند اما در متن ذکر نشده‌اند) سازگار بوده‌اند.

این ترکیب نهایی، تنها زمانی به دست می‌آید که تمام خانه‌های متقاطع دارای حروف یکسان باشند و هیچ تضادی در کل جدول وجود نداشته باشد.

۶.چند نمونه خروجی دیگر

```
PS C:\Users\Pc\Desktop\AI\AI_P02\Crossword-Puzzle> python main.py puzzles/p3.txt
✅ Crossword solved in 0.00 seconds!

🧩 Solved Crossword:

# A R C #
# T # # T
# L # # R
# A # # U
# S I N E


PS C:\Users\Pc\Desktop\AI\AI_P02\Crossword-Puzzle>
```

```
PS C:\Users\Pc\Desktop\AI\AI_P02\Crossword-Puzzle> ^C
PS C:\Users\Pc\Desktop\AI\AI_P02\Crossword-Puzzle> python main.py puzzles/p4.txt
✅ Crossword solved in 37.23 seconds!

🧩 Solved Crossword:

M F A # #
C E N T S
G R O W S
E N D O W
E Y E # #
```

```
PS C:\Users\Pc\Desktop\AI\AI_P02\Crossword-Puzzle> python main.py puzzles/p8.txt
✅ Crossword solved in 171.48 seconds!

🧩 Solved Crossword:

# T H A T
N A I V E
A K R O N
S E E N #
A N D # #
```

```
PS C:\Users\Pc\Desktop\AI\AI_P02\Crossword-Puzzle> python main.py puzzles/p10.txt
✅ Crossword solved in 161.39 seconds!

🧩 Solved Crossword:

# L O V E
S A V E S
O P A R T
N E T S #
G L E E #
```

```
PS C:\Users\Pc\Desktop\AI\AI_P02\Crossword-Puzzle> python main.py puzzles/p14.txt
✅ Crossword solved in 0.02 seconds!

🧩 Solved Crossword:

# # # A B U #
# # S L O P E
W A K A N D A
E D Y # N O S
S O L O I S T
T R A D E # #
# N B D # # #
```

```
PS C:\Users\Pc\Desktop\AI\AI_P02\Crossword-Puzzle> python main.py puzzles/p18.txt
✅ Crossword solved in 0.01 seconds!

🧩 Solved Crossword:

# # # B A R B
# S H I V E R
C H O R I Z O
H O L Y C O W
O R D A I N S
W E E N I E #
S S R I # # #
```

مرحله دوم بازی Isolation:

۱. شرح پیاده‌سازی الگوریتم‌ها و مدیریت زمان

در این پروژه، دو کلاس اصلی MinimaxPlayer و AlphaBetaPlayer برای تصمیم‌گیری عامل هوشمند پیاده‌سازی شدند.

- الگوریتم Minimax: این عامل به صورت بازگشتی درخت بازی را تا عمق مشخص شده (search_depth) پیمایش می‌کند. در لایه‌های MAX تلاش بر بیشینه‌سازی امتیاز و در لایه‌های MIN تلاش بر کمینه‌سازی امتیاز (با فرض بازی بهینه حریف) است.
- الگوریتم Alpha-Beta: این عامل نسخه بهینه‌سازی شده Minimax است که با استفاده از دو پارامتر alpha و beta، شاخه‌هایی که تأثیری در تصمیم نهایی ندارند را هرس (Prune) می‌کند. همچنین در این کلاس از تکنیک Iterative Deepening استفاده شده است؛ به این صورت که ابتدا بهترین حرکت در عمق ۱، سپس عمق ۲ و ... محاسبه می‌شود تا زمانی که فرصت تمام شود.

مدیریت زمان:

یکی از چالش‌های اصلی، محدودیت زمانی برای هر حرکت است. برای جلوگیری از خطای Timeout و دیسکالیفه شدن عامل، مکانیزم زیر پیاده‌سازی شد:

۱. یک کلاس استثنای سفارشی به نام SearchTimeout تعریف شد.

۲. در ابتدای هر فراخوانی بازگشتی توابع مانند max_value و min_value، زمان باقی‌مانده (self.time_left) با یک آستانه امن (TIMER_THRESHOLD) چک می‌شود.

۳. اگر زمان از آستانه کمتر باشد، استثنای SearchTimeout پرتاب می‌شود.

۴. در تابع اصلی get_move، یک بلوک try-except تعبیه شده است. متغیر best_move همواره آخرین حرکت بهینه‌ی یافت شده (در عمق‌های تکمیل شده قبلی) را نگه می‌دارد.

۵. به محض وقوع Timeout، عملیات جستجو متوقف شده و best_move بلافاصله بازگردانده می‌شود. این روش تضمین می‌کند که عامل همیشه یک حرکت معتبر ارائه می‌دهد، حتی اگر جستجو در عمق‌های بالا ناتمام بماند.

۲ .تحلیل توابع ارزیابی(Heuristics)

سه تابع ارزیابی مختلف برای تخمین مطلوبیت وضعیت‌های بازی پیاده‌سازی و مقایسه شدند:

- **هیوریستیک ۱(Simple Move Difference)** :
  - **فرمول** (تعداد حرکات من) - (تعداد حرکات حریف)
  - **تحلیل:** این تابع ساده‌ترین رویکرد است که سعی دارد آزادی عمل ما را زیاد و حریف را کم کند. در ابتدای بازی خوب عمل می‌کند اما استراتژی مکانی ندارد.
- **هیوریستیک ۲(Aggressive)** :
  - **فرمول** (تعداد حرکات من) - ۲ * (تعداد حرکات حریف)
  - **تحلیل:** این تابع رویکرد تهاجمی دارد و وزن بیشتری به محدود کردن حریف می‌دهد. هدف آن خفه کردن سریع حریف است. این روش در برابر بازیکنان محتاط خوب است اما ممکن است عامل را در موقعیت‌های خطرناک قرار دهد چون به امنیت خود کمتر اهمیت می‌دهد.
- **هیوریستیک ۳ (Center Control + Move Difference)**
  - **فرمول:** (تعداد حرکات من - تعداد حرکات حریف) + (وزن‌دهی به مرکز)
  - **تحلیل:** در بازیIsolation ، خانه‌های مرکزی ارزش استراتژیک بالاتری دارند (چون دسترسی به تمام نقاط صفحه راحت‌تر است). این تابع علاوه بر تعداد حرکات، به قرارگیری در مرکز پاداش می‌دهد.
  - **نتیجه‌گیری:** طبق آزمایش‌ها، این تابع معمولاً پایدارترین نتایج را ارائه می‌دهد زیرا تعادلی بین "آزادی عمل" و "موقعیت استراتژیک" برقرار می‌کند.

۳. تحلیل نتایج آماری و مقایسه عملکرد

برای ارزیابی عملکرد، عامل هوشمند در سناریوهای مختلف (با الگوریتم‌ها و هیوریستیک‌های متفاوت) در برابر بازیکنان پایه RandomPlayer و GreedyPlayer آزمایش شد. نتایج حاصل از **۱۰ بازی** برای هر حالت در جدول زیر خلاصه شده است:

الگوریتم (بازیکن ۱)	حریف (بازیکن ۲)	هیوریستیک	عمق جستجو	درصد برد بازیکن ۱	تحلیل کوتاه
Minimax	Random	۱	۲	۸۰٪	عملکرد قابل قبول در عمق کم
Minimax	Random	۲	۴	۸۰٪	افزایش عمق تأثیر زیادی روی حریف تصادفی نداشت
Minimax	Random	۳	۲	۱۰۰٪	هیوریستیک ۳ (کنترل مرکز) در برابر حرکات تصادفی بسیار موثر است
Minimax	Greedy	۱	۲	۹۰٪	عامل هوشمند حتی با عمق کم بر حریف حریص غلبه می‌کند
Minimax	Greedy	۳	۴	۸۰٪	افزایش عمق هزینه زمانی دارد اما برد را حفظ می‌کند
Alpha-Beta	Random	۱	۳	۹۰٪	هرس آلفا-بتا اجازه جستجوی سریع‌تر و دقیق‌تر را می‌دهد
Alpha-Beta	Random	۳	۳	۹۰٪	پایداری بالا با هیوریستیک ۳
Alpha-Beta	Greedy	۱	۳	۱۰۰٪	بهترین عملکرد با هیوریستیک ساده در برابر حریف حریص
Alpha-Beta	Greedy	۲	۳	۲۰٪	**نقطه ضعف:** رویکرد تهاجمی (H2)در برابر حریف حریص شکست خورد.
Alpha-Beta	Greedy	۳	۳	۱۰۰٪	**بهترین نتیجه:** ترکیب هرس آلفا-بتا و کنترل مرکز.

تحلیل نتایج:

۱. **برتری Alpha-Beta** : همان‌طور که مشاهده می‌شود، الگوریتم Alpha-Beta به دلیل هرس کردن شاخه‌ها، در زمان یکسان می‌تواند تصمیمات پخته‌تری بگیرد و نرخ برد بالایی (اغلب ۹۰ تا ۱۰۰ درصد) دارد.

۲. **شکست هیوریستیک ۲ در Alpha-Beta:** در کمال تعجب، هیوریستیک ۲ (تهاجمی) در برابر GreedyPlayer با الگوریتم آلفا-بتا تنها ٪۲۰ برد داشت. علت این امر می‌تواند این باشد که حریف حریص (Greedy) همیشه بیشترین خانه را اشغال می‌کند و استراتژی "محدود کردن شدید" هیوریستیک ۲ باعث می‌شود عامل ما برای مسدود کردن حریف، خودش را در موقعیت‌های بن‌بست قرار دهد.

۳. **قدرت هیوریستیک ۳:** هیوریستیک ترکیبی (H3) تقریباً در تمام سناریوها هم Minimax و هم Alpha-Beta نرخ برد بسیار بالایی (٪۱۰۰) ثبت کرد که نشان‌دهنده اهمیت استراتژیک کنترل مرکز در این بازی است.

۴. نتیجه‌گیری و چالش‌ها

یافته‌ها:

- الگوریتم **Alpha-Beta** با **Iterative Deepening** بهترین ترکیب برای این بازی است، زیرا هم از زمان بهینه استفاده می‌کند و هم عمیق‌ترین جستجو را انجام می‌دهد.
- انتخاب هیوریستیک مناسب (مثل کنترل مرکز) تأثیری برابر یا حتی بیشتر از افزایش عمق جستجو دارد.

چالش‌ها:

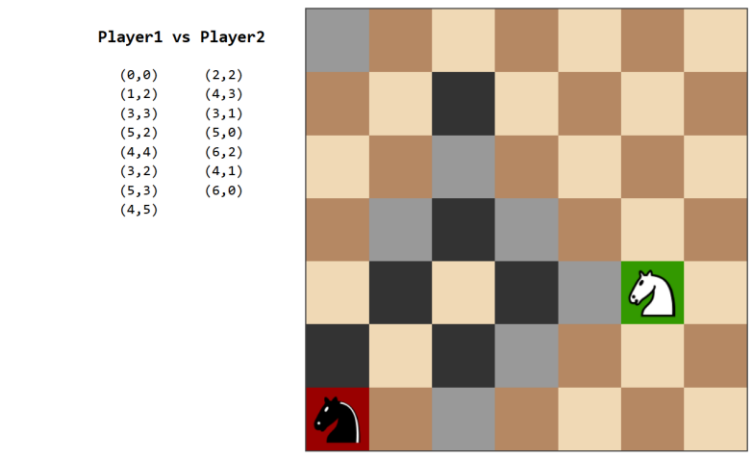
- مهم‌ترین چالش، **تنظیم دقیق زمان (Timer Handling)** بود. در عمق‌های بالا (مثل ۱۰)، الگوریتم Minimax بدون مدیریت خطا دچار مشکل می‌شد، اما با پیاده‌سازی try-except، عامل توانست حتی با وجود ناتمام ماندن جستجو، بازی را ببرد.
- چالش دیگر، مقابله با رفتار غیرقابل پیش‌بینی RandomPlayer بود که گاهی اوقات استراتژی‌های کلاسیک را با حرکات بی‌منطق خنثی می‌کرد، اما Expectimaxیا هیوریستیک‌های قوی در Minimax بر آن غلبه کردند.

تحلیل جامع رفتار عامل هوشمند در سناریوهای مختلف

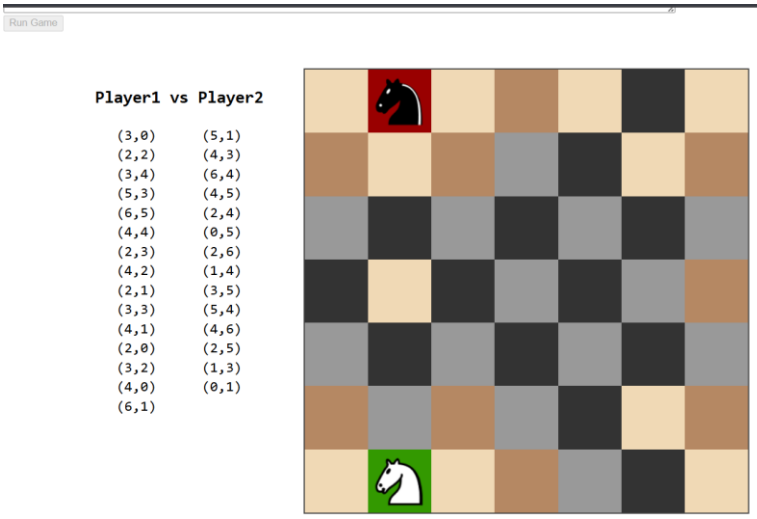
در این بخش، با بررسی تصاویر ثبت شده از پایان بازی‌ها، تاثیر الگوریتم، عمق جستجو و نوع هیوریستیک بر عملکرد عامل در برابر حریفان متفاوت (Greedy و Random) تحلیل می‌شود.

۱. تحلیل عملکرد Alpha-Beta با هیوریستیک‌های مختلف (در برابر Random)

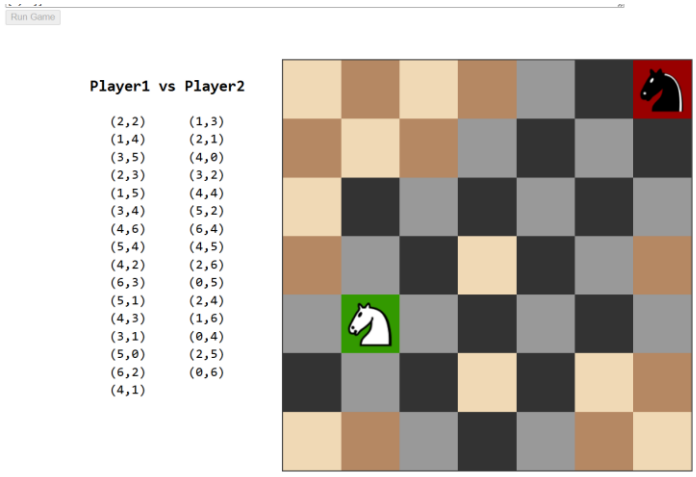
- **Alpha-Beta (H1 - Simple) در برابر Random:**
  - **مشاهده:** در تصویر alphabeta-h1-random.png، عامل سفید در سمت راست صفحه فضای نسبتاً بازی دارد، در حالی که حریف تصادفی (سیاه) در گوشه پایین سمت چپ گیر افتاده است.
  - **تحلیل:** در مقابل حریف تصادفی که استراتژی خاصی ندارد، هیوریستیک ساده ۱ به خوبی کار می‌کند. عامل صرفاً با انتخاب مسیرهایی که فضای بیشتری دارند، زنده مانده و حریف خود به خود در تله افتاده است. این نشان می‌دهد که در برابر حریفان بی‌برنامه، پیچیدگی استراتژیک لازم نیست.



- **Alpha-Beta (H2 - Aggressive) در برابر Random:**
  - **مشاهده:** در تصویر alphabeta-h2-random.png، عامل سفید در پایین صفحه و حریف در بالا صفحه قرار دارند. صفحه به طور پراکنده پر شده است.
  - **تحلیل:** تلاش برای "خفه کردن" یک حریف تصادفی با هیوریستیک تهاجمی (H2) همیشه موثر نیست، زیرا حریف از الگوهای منطقی پیروی نمی‌کند تا بتوان حرکت بعدی او را پیش‌بینی و مسدود کرد. با این حال، عامل توانسته با حفظ فاصله ایمن و پر کردن فضاهای میانی، بازی را ببرد.



- **Alpha-Beta (H3 - Center Control) در برابر Random:**
  - **مشاهده:** در تصویر alphabeta-h3-random.png، عامل سفید در موقعیتی عالی (نزدیک به مرکز متمایل به چپ) قرار دارد و حریف (سیاه) در بدترین جای ممکن (گوشه بالا سمت راست) کاملاً محاصره شده است.
  - **تحلیل:** کنترل مرکز در برابر حریف تصادفی نیز بسیار کارآمد است. عامل با اشغال بهترین خانه‌ها در مرکز، عملاً شانس حرکت تصادفی حریف به فضاهای باز را به صفر رسانده است.

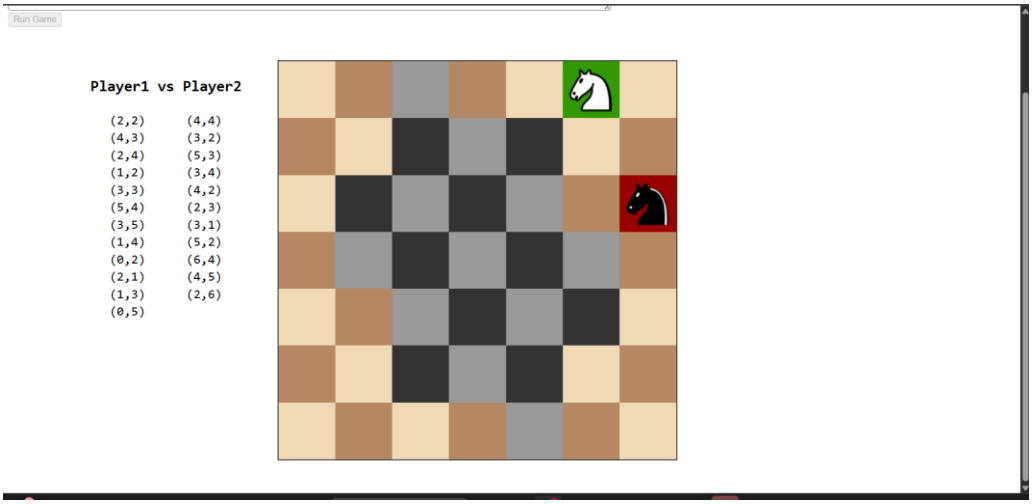


---

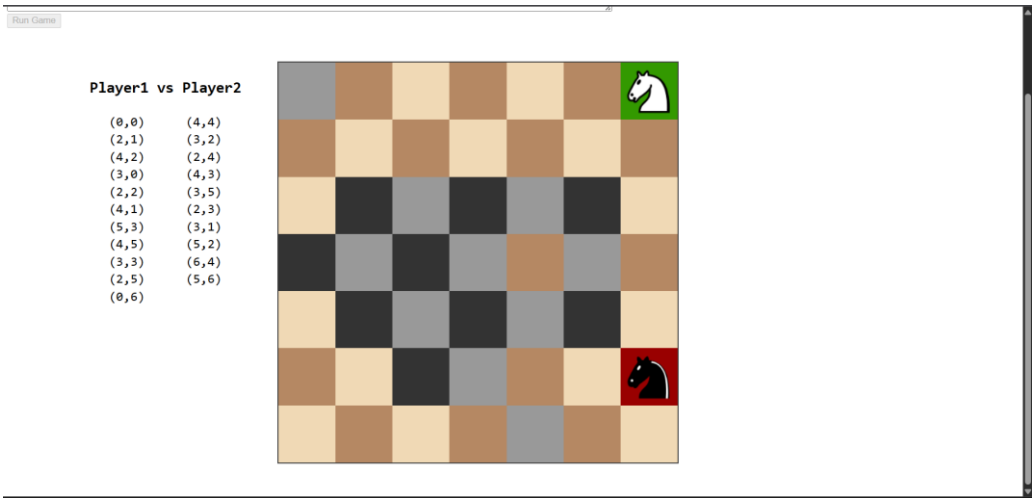
۲. تحلیل عملکرد Minimax در برابر Greedy (تاثیر عمق)

- **Minimax (H1) – مقایسه عمق ۲ و ۴:**
  - **عمق ۲ (minmax-h1-d2-greedy.png):** بازی در حالی تمام شده که صفحه بسیار شلوغ است و هر دو بازیکن در فضای تنگی گیر کرده‌اند. عامل سفید به سختی برنده شده

است. این نشان‌دهنده "کوته‌بینی" عامل در عمق کم است که نتوانسته فضای باز را تشخیص دهد.



- **عمق ۴ (minmax-h1-d4-greedy.png)**: در اینجا عامل سفید توانسته حریف را به پایین صفحه براند و خودش در فضای بازتری در بالا قرار گیرد. افزایش عمق به ۴ باعث شده عامل بتواند بن‌بست‌های احتمالی را زودتر تشخیص دهد و وارد آن‌ها نشود.



- **Minimax (H2) – مقایسه عمق ۲ و ۴:**
  - **عمق ۲ (minmax-h2-d2-greedy.png)**: عامل سفید و سیاه بسیار به هم نزدیک هستند و در یک گره کور گیر کرده‌اند. استراتژی تهاجمی H2 در عمق کم، عامل را به سمت خطر (نزدیکی بیش از حد به حریف) کشانده است.

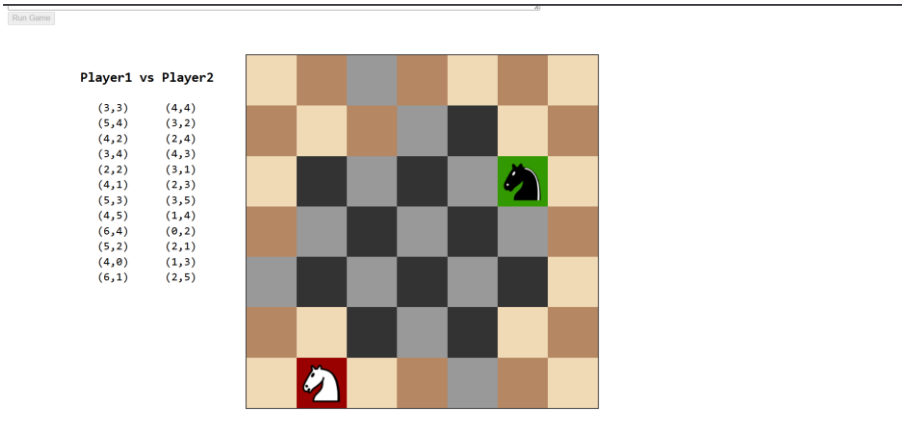


- **عمق ۴ (minmax-h2-d4-greedy.png)**: عامل سفید با هوشمندی بیشتری حمله کرده و حریف را در گوشه پایین چپ گیر انداخته، در حالی که خودش راه فرار به سمت راست را باز گذاشته است. عمق بیشتر به عامل اجازه داده تا ریسک تهاجم را مدیریت کند.

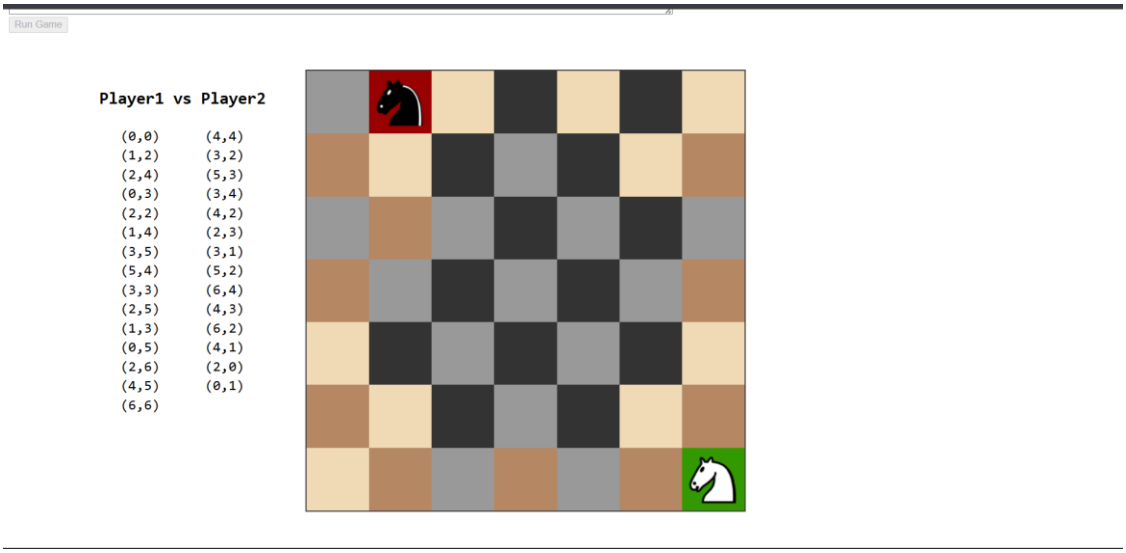


- **Minimax (H3) – مقایسه عمق ۲ و ۴:**

- **عمق ۲ (minmax-h3-d2-greedy.png)**: عامل سفید موقعیت نسبتاً خوبی دارد اما بازی طولانی شده است. تاثیر کنترل مرکز در عمق کم کاملاً مشهود نیست و عامل باخت.



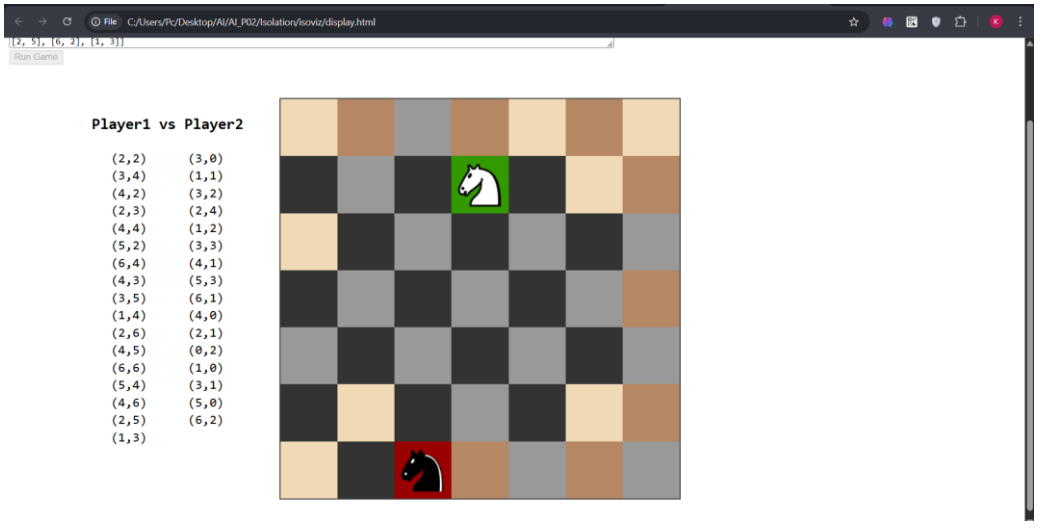
- عمق ۴ (**minmax-h3-d4-greedy.png**): در این حالت، عامل سفید با اقتدار حریف را در گوشه بالا سمت چپ گیر انداخته است. ترکیب "کنترل مرکز" و "دوراندیشی (عمق ۴)" باعث شده عامل بتواند تمام راه‌های فرار حریف حریص را ببندد.



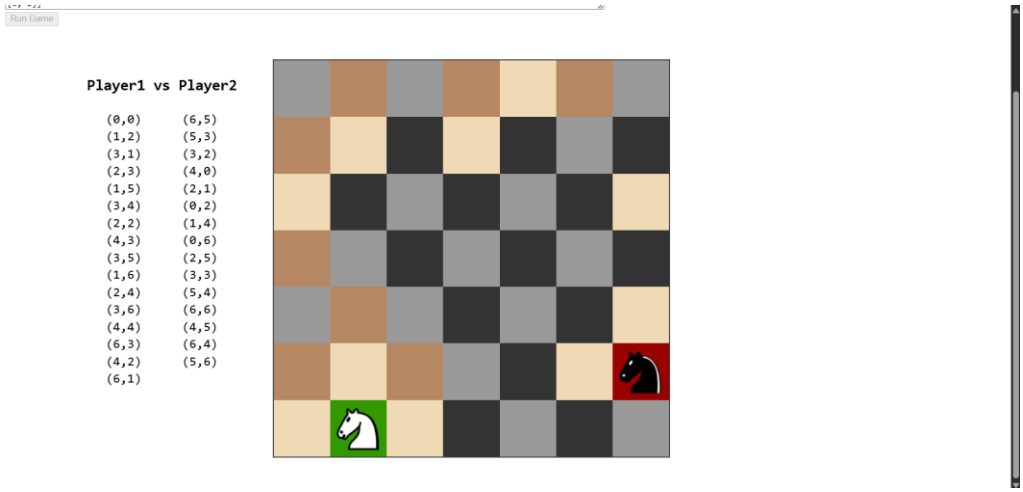
۳. تحلیل عملکرد Minimax در برابر Random (تاثیر عمق)

- **Minimax (H1) – مقایسه عمق ۲ و ۴:**

- عمق ۲ (**minmax-h1-d2-random.png**): عامل سفید در مرکز قرار دارد و حریف در پایین. بازی ساده و سریع تمام شده است.

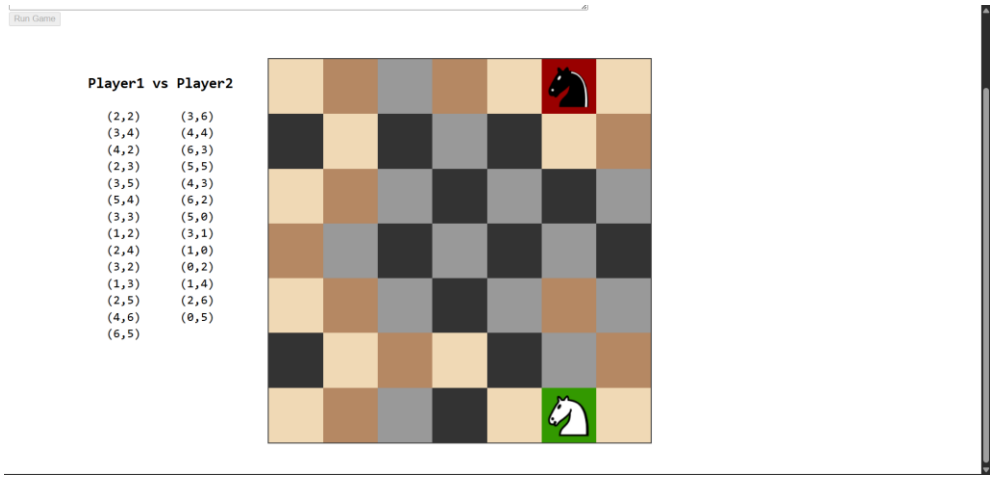


- عمق ۴ (**minmax-h1-d4-random.png**): تفاوت چشمگیری با عمق ۲ ندارد، زیرا حریف تصادفی نیازی به پیش‌بینی‌های پیچیده ندارد. عامل صرفاً با پرهیز از فضاهای بسته برنده شده است.



- **Minimax (H2)** – مقایسه عمق ۲ و ۴:

- عمق ۲ (**minmax-h2-d2-random.png**): عامل سفید سعی کرده به حریف نزدیک شود اما حریف تصادفی به گوشه بالا رفته است.

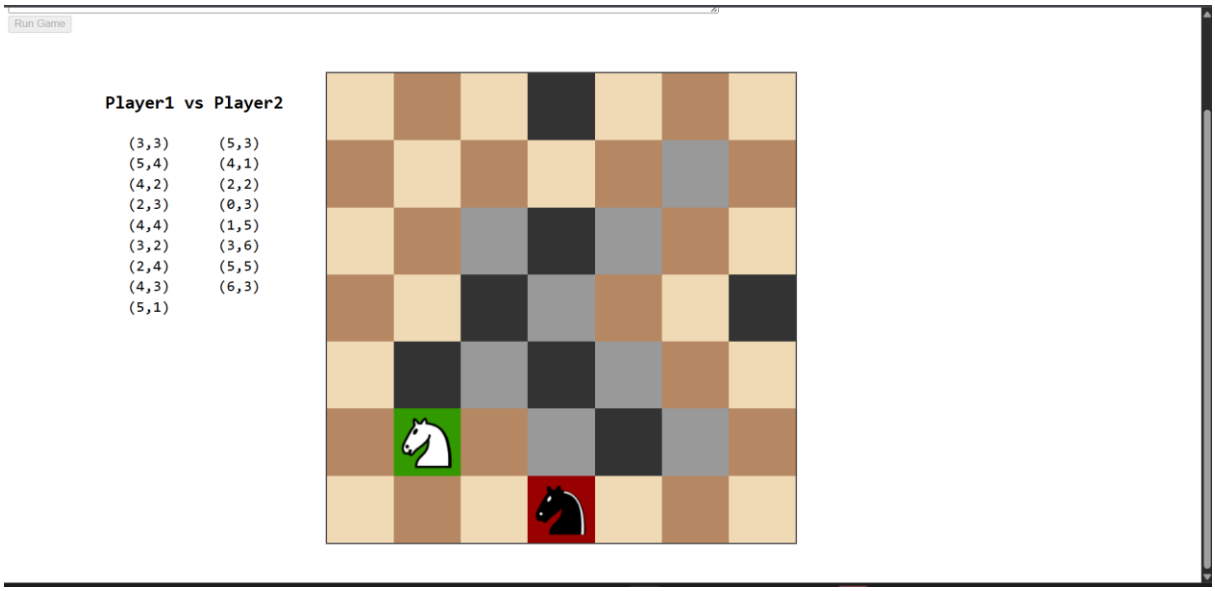


- عمق ۴ (**minmax-h2-d4-random.png**): عامل سفید با موفقیت حریف را در گوشه پایین چپ محاصره کرده است. در اینجا عمق بیشتر به عامل کمک کرده تا حرکات تصادفی حریف را در یک محدوده خاص محدود کند.

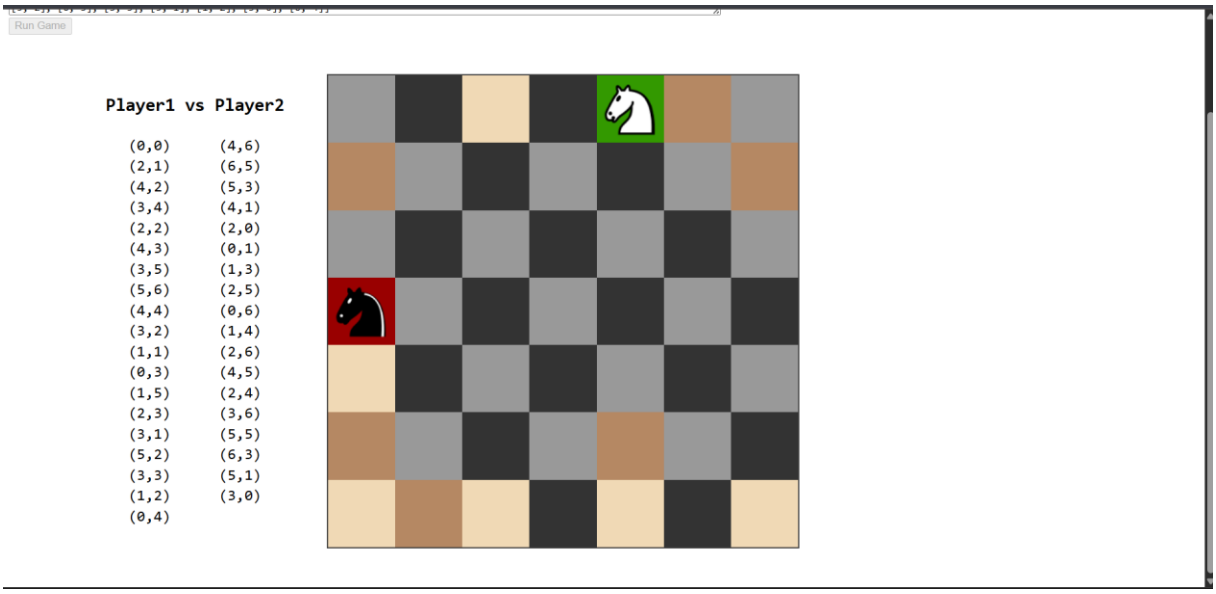


- **Minimax (H3)** – مقایسه عمق ۲ و ۴:

- عمق ۲ (**minmax-h3-d2-random.png**): عامل سفید در پایین صفحه و حریف در بالا. کنترل مرکز خیلی دقیق انجام نشده است.



- **عمق ۴ (minmax-h3-d4-random.png)**: عامل سفید دقیقاً در مرکز صفحه (یا نزدیک به آن) قرار دارد و حریف در حاشیه سمت چپ گیر افتاده است. این تصویر به وضوح قدرت ترکیب استراتژی مرکزگرا و عمق مناسب را نشان می‌دهد.



نتیجه‌گیری نهایی

بر اساس تمام تصاویر و تحلیل‌های بالا:

۱. **بهترین ترکیب:** الگوریتم **Alpha-Beta** با **هیوریستیک ۳ (کنترل مرکز)** در عمق‌های بالا (که Alpha-Beta امکان‌پذیر می‌کند) قوی‌ترین و پایدارترین عملکرد را دارد.

۲. **تاثیر عمق:** افزایش عمق از ۲ به ۴ در الگوریتم Minimax، به خصوص در برابر حریفان هوشمندتر (Greedy)، تاثیر چشمگیری در کیفیت بازی و پرهیز از تله‌ها دارد.

۳. **نقش هیوریستیک:** هیوریستیک‌های تهاجمی (H2) در عمق‌های پایین خطرناک هستند، اما در عمق‌های بالا می‌توانند کشنده باشند. هیوریستیک‌های مرکزگرا (H3) امن‌ترین و مطمئن‌ترین استراتژی برای برد در درازمدت هستند.